

Lab 4 Group Work: Collaborating on a Shared GitHub Repository

192-211 Automated Software Testing • Groups of 3–5 students • 20 marks

Estimated time: About 3 hours

In previous labs, you worked with Git and pytest on your own computer. In this activity, you will practise a new and important skill: **working together in a shared GitHub repository**.

This is the main focus of the assignment and where most of the marks come from.

The testing tasks are intentionally small. Each student only needs to create:

- One pytest fixture
- Two simple tests

You can reuse patterns from earlier labs. If you spend a lot of time debugging pytest, you are probably focusing on the wrong part of the assignment. The main challenge is learning how to use **Git and GitHub collaboratively**.

Deliverable

Submit **one public GitHub repository** containing:

- Contributions from every group member
- A completed `README.md`
- A test suite that passes successfully

Part 0: Understanding Where Your Code Is

Git stores your work in several places. Many beginner mistakes happen because students lose track of where their changes currently are.

Whenever something seems confusing, think about which of the following four locations your code is in.

Location	Description	How to Check
Working Directory	Files you are currently editing	<code>git status</code>
Staging Area	Files selected for the next commit	<code>git status</code>
Local Repository	Commits saved on your computer	<code>git log --oneline</code>
GitHub Repository (Remote)	Commits uploaded to GitHub and visible to teammates	GitHub repository page

Important Ideas

1. Saving Is Not the Same as Committing

Saving a file only updates it on your computer. Git does not automatically create a commit when you save.

2. Committing Is Not the Same as Pushing

A commit exists only in your local repository. Your teammates cannot see your work until you upload it to GitHub using:

```
git push
```

3. Everyone Has Their Own Copy of the Repository

Each team member has a complete local copy of the project.

Because of this, your commit history may temporarily differ from your teammates' histories. This is normal. To download the latest changes from the shared repository, use:

```
git pull
```

Tip: Run `git status` frequently during this lab. It shows what has changed, what is staged, and often suggests the next command to run.

Round 1: Create and Join the Repository

Repository Owner (One Person Only)

Create a new repository on GitHub.

1. Click **New Repository**.
2. Name it:

```
lab04-<group-name>
```

3. Set the repository to **Public**.
4. Select **Add a README file**.
5. Click **Create Repository**.

Next, add all teammates as collaborators:

Settings → **Collaborators** → **Add people**

Use each teammate's GitHub username when sending invitations.

Because the repository is public, anyone can view its contents. Do not upload private or personal information.

Other Group Members

1. Accept the collaboration invitation from GitHub.
2. Do **not** click **Fork**.

A fork creates a separate copy of the repository under your own account. Changes made there will not automatically become part of your group's repository.

Instead, wait for access to the shared repository and clone it directly.

Everyone (Including the Owner)

Clone the repository:

```
git clone https://github.com/<owner>/lab04-<group>.git
cd lab04-<group>
```

Configure Git so that your commits are correctly attributed to you:

```
git config user.name "Your Name"
git config user.email "your-email@example.com"
```

Verify that everything is working:

```
git log --oneline
git status
```

You should see:

```
nothing to commit, working tree clean
```

If Git Asks for a Password

GitHub no longer accepts account passwords for Git operations.

Choose one of the following options:

- Install GitHub CLI and run `gh auth login`.
- Create a Personal Access Token and use it instead of a password when prompted.

You only need to complete this setup once.

Repository Owner: Initial Setup

Before anyone else starts committing, create the `.gitignore` file and add the `bank.py` module.

```
cat > .gitignore << 'EOF'
venv/
__pycache__/
.pytest_cache/
*.pyc
EOF

cat > bank.py << 'PY'
class BankAccount:
def __init__(self, balance=0):
self.balance = balance

def deposit(self, amount):
self.balance += amount
return self.balance

def withdraw(self, amount):
if amount > self.balance:
raise ValueError("Insufficient funds")
self.balance -= amount
return self.balance
PY

git add .gitignore bank.py
git commit -m "chore: add gitignore and bank module"
git push
```

The `.gitignore` file prevents temporary and generated files from being uploaded to GitHub. Without it, someone may accidentally commit unnecessary files such as virtual environments or cache folders, making the repository difficult to manage.

Everyone Else

After the repository owner has pushed the changes, run:

```
git pull
```

You should now see:

```
.gitignore
bank.py
```

in your project folder.

Next, set up a virtual environment and install pytest:

```
python3 -m venv venv
source venv/bin/activate
pip install pytest -q
```

Checkpoint 1

Before continuing, every team member should run:

```
git log --oneline
```

Everyone should see the same two commits.

If a team member sees different commits, they may have cloned the wrong repository. Verify the repository URL using:

```
git remote -v
```

The URL should point to the repository owner's account, not your own.

Round 2: Create and Push Your Own Test File

In this round, each team member creates a different file. Because everyone is working on separate files, there should be no conflicts at this stage.

Member	Your File	Task
A	test_deposit.py	Create a fixture <code>account()</code> that returns <code>BankAccount(100)</code> and write two deposit tests
B	test_withdraw.py	Create a fixture <code>account()</code> that returns <code>BankAccount(100)</code> , one withdrawal test, and one overdraft test using <code>pytest.raises(ValueError)</code>
C	test_teardown.py	Create a <code>yield</code> fixture that prints <code>[setup]</code> before and <code>[teardown]</code> after, then write two tests using it
D	test_shared.py	Write two tests using <code>funded_account</code> from <code>conf.test.py</code>

Member	Your File	Task
E	<code>conftest.py</code>	Create a fixture <code>funded_account()</code> that returns <code>BankAccount(1000)</code>

Three-member groups: Member A also completes E.

Four-member groups: Member A also completes E.

Five-member groups: One task per member.

Member A or E should push `conftest.py` early because Member D depends on it.

Example structure:

```
import pytest
from bank import BankAccount

@pytest.fixture
def account():
    return BankAccount(100)

def test_deposit_increases_balance(account):
    account.deposit(50)
    assert account.balance == 150
```

Your Workflow

For the rest of the lab, follow this process whenever you make changes:

```
git pull
pytest -v
git status
git add <your-file>
git commit -m "<meaningful message>"
git push
```

Why this order?

1. Download your teammates' latest work.
2. Make sure your tests pass.
3. Check what has changed.
4. Stage only the file you worked on.
5. Create a commit with a meaningful message.
6. Push your work so the rest of the team can see it.

If Your Push Is Rejected

When several people are working on the same repository, it is common for a push to be rejected.

You may see a message similar to this:

```
! [rejected] main -> main (fetch first)
error: failed to push some refs
```

This usually means that another team member pushed changes to GitHub before you did. As a result, your local copy of the repository is no longer up to date.

Do not panic. This is a normal part of collaborative software development.

To fix the problem:

```
git pull
git push
```

The `git pull` command downloads the latest changes from the shared repository and merges them into your local copy. Once your repository is up to date, you can try pushing again.

In some cases, Git may report a merge conflict during `git pull`. If that happens, resolve the conflict first, commit the resolution, and then run `git push` again.

Remember: A rejected push does not mean your work is lost. Git is simply preventing you from accidentally overwriting someone else's changes.

Writing Good Commit Messages

A commit message should explain what was achieved or what is now working. This makes the project history easier for your teammates to understand.

Good examples:

```
test: account fixture for deposit tests
test: overdraft raises ValueError
test: add tests for shared fixture
test: yield fixture prints setup and teardown
```

Poor examples:

```
update
fix
final
asdf
test
```

A good commit message should allow someone unfamiliar with your code to understand the purpose of the change by reading the project history.

Checkpoint 2

Before moving on, every team member should run:

```
git pull
pytest -v
```

All tests should pass successfully on every group member's computer, not just the tests they created themselves.

At this point:

- Every team member should have at least one commit in the repository.
- Everyone should be able to see all test files created by the group.
- The complete test suite should pass without errors.

Round 3: Create and Resolve a Merge Conflict

This is the most important part of the lab.

A merge conflict is not a mistake or a failure. It simply means that Git has detected changes to the same part of a file and cannot determine automatically which version should be kept.

Learning how to resolve merge conflicts is an essential skill because collaborative software projects frequently encounter them.

Create the Conflict

All group members should edit `README.md` at roughly the same time.

Add your own row to the following table:

```
## Who Did What

| Member | GitHub Username | File |
|---|---|---|
| Your Name | your-username | test_deposit.py |
```

Then run:

```
git add README.md
git commit -m "docs: add my row to the who-did-what table"
git pull
git push
```

Usually, one team member will push successfully first. Other members may receive a merge conflict because they modified the same section of the file.

Understanding the Conflict Markers

When a conflict occurs, Git inserts special markers into the file:

```
<<<<<<< HEAD
| Mai | mai-dev | test_withdraw.py |
=====
| Nok | nok-codes | test_deposit.py |
>>>>>>> 3f2a9c1
```

These markers show two competing versions:

- Everything between <<<<<<< HEAD and ===== is your local version.
- Everything between ===== and >>>>>>> came from another commit.

Git stops the merge because it does not know which version should remain in the final file.

Resolving the Conflict

Open the file and read both versions carefully.

In this activity, both rows should be kept because each row belongs to a different group member.

After removing the conflict markers, the table should look like this:

```
| Mai | mai-dev | test_withdraw.py |
| Nok | nok-codes | test_deposit.py |
```

Save the file and complete the merge:

```
git add README.md
git commit -m "docs: resolve README merge conflict"
git push
```

By staging the file and creating a commit, you are telling Git that the conflict has been resolved.

If Something Goes Wrong

If you become stuck during the merge process, you can cancel the merge and return to the previous state:

```
git merge --abort
```

After that, review the situation and try again.

Checkpoint 3

Before continuing:

- Every group member's row should appear in the table.

- No conflict markers should remain in the file.
- The repository history should show that a merge took place.

Search the file for any remaining conflict markers:

```
<<<<<<<
=====
>>>>>>>
```

If any are still present, the conflict has not been fully resolved.

Documenting the Conflict

Add a section called **Our Merge Conflict** to `README.md`.

Include:

1. The conflict markers that your team encountered.
2. Which lines were kept in the final version.
3. A brief explanation of why Git could not resolve the conflict automatically.

This section is part of the assessment and demonstrates that your group successfully experienced and resolved a real merge conflict.

Round 4: Inspecting and Reverting Changes

Git provides several commands that help you understand the current state and history of a repository.

Try each of the following commands:

```
git status
git diff
git log --oneline --graph
git show HEAD
```

Take a few minutes to examine their output and understand what information each command provides.

What These Commands Do

```
git status
```

Shows:

- Modified files
- Staged files

- Untracked files
- Suggestions for next commands

```
git diff
```

Shows the exact lines that have changed but have not yet been staged.

```
git log --oneline --graph
```

Shows a compact view of the project's history, including merges.

```
git show HEAD
```

Shows the changes introduced by the most recent commit.

Restoring a File

Create a temporary change:

```
echo "this is rubbish" >> test_deposit.py
```

View the change:

```
git diff
```

Now discard it:

```
git restore test_deposit.py
```

Verify that the change is gone:

```
git diff
```

The file should now match its most recently committed version.

`git restore` only removes uncommitted changes. Once those changes are discarded, Git cannot recover them. Commit important work regularly.

Round 5: Verify Everyone's Contribution

Before submitting, confirm that every group member has contributed to the repository.

Run:

```
git pull
git shortlog -sn
git log --oneline --graph
pytest -v
pytest -v -s
```

Checking Contributions

The command:

```
git shortlog -sn
```

displays the number of commits made by each contributor.

Example:

```
5 Alice
4 Bob
4 Charlie
```

Review the output carefully.

If your name is missing or displayed incorrectly, it usually means that `git config user.name` was not set correctly before making commits.

If this happens, update your Git configuration and make additional commits using the correct name.

You can also verify contributions on GitHub through:

Insights → Contributors

Final Verification

Before submission, ensure that:

- All members appear in `git shortlog -sn`.
- The repository contains commits from every member.
- All tests pass.
- The README is complete.
- No merge conflict markers remain in any file.

What Your README Must Contain

Your README .md should contain the following sections:

1. Group Name

Include your group name at the top of the document.

2. Who Did What Table

Include one row for every team member.

3. Our Merge Conflict

Describe:

- The conflict markers encountered
- The final decision made by the team
- Why Git could not automatically resolve the conflict

4. Git Contribution Summary

Paste the output of:

```
git shortlog -sn
```

5. Reflection Questions

Answer each question in one or two sentences:

1. Why was your push rejected, and how did you fix it?
2. Why could Git not resolve the README conflict automatically?
3. What is the difference between committing and pushing?
4. How do fixtures reduce duplicated setup code in tests?

Finally, submit the GitHub repository URL.

Git Command Reference

Command	Purpose
<code>git clone <url></code>	Create a local copy of a repository
<code>git config user.name "..."</code>	Set the name attached to your commits
<code>git status</code>	View changes, staged files, and suggestions

Command	Purpose
<code>git diff</code>	View uncommitted changes
<code>git add <file></code>	Stage a file for the next commit
<code>git commit -m "..."</code>	Save a snapshot in your local repository
<code>git push</code>	Upload commits to GitHub
<code>git pull</code>	Download teammates' latest changes
<code>git log --oneline --graph</code>	View commit history
<code>git show HEAD</code>	Show the latest commit in detail
<code>git restore <file></code>	Discard uncommitted changes
<code>git merge --abort</code>	Cancel an unfinished merge
<code>git shortlog -sn</code>	Count commits by contributor
<code>git remote -v</code>	Show connected repositories

Marking (20 Marks)

Requirement	Marks
Every member has at least three commits, correctly attributed using <code>git shortlog -sn</code>	5
Round 3 merge conflict occurred and was resolved correctly	4
Repository setup is correct (<code>.gitignore</code> , collaborators, no unnecessary files committed)	3
Commit messages clearly describe completed work	2
Tests completed as required, including use of <code>confest.py</code>	4
README contains all required sections	2

A group member with no commits will receive no marks for this lab, regardless of the group's overall score. Commit regularly throughout the activity so that your contributions can be verified.